

Trimestre 26-P — Reporte de Servicio Social

Programa: Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo **Alumno:** Oscar David González Pintor — Matrícula: 2173071702 **Licenciatura:** Ingeniería en Computación — UAM Cuajimalpa **Responsable del proyecto:** Dra. Alicia Montserrat Alvarado González (Núm. económico: 41051) **Periodo reportado:** 1 de abril de 2026 — 30 de junio de 2026 (Meses 1, 2 y 3)

Introducción

El presente reporte trimestral documenta las actividades realizadas durante los primeros tres meses del servicio social, inscrito bajo el programa **Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo**. Este proyecto busca dotar a un brazo robótico simulado de una arquitectura de cómputo inspirada en los principios neurocognitivos de las neuronas espejo: regiones corticales especializadas que se activan tanto al ejecutar una acción motora como al observar a otro agente ejecutarla, estableciendo la base neurológica de la imitación, el aprendizaje y la predicción motora.

El sustrato físico de la arquitectura es el brazo robótico antropomórfico **Xolobot**, un manipulador de 21 grados de libertad con una mano de cinco dedos. El trabajo de este trimestre se centró en migrar y modernizar el entorno de simulación del Xolobot — necesario para que las cortezas cognitivas pudieran ejecutarse de forma reproducible — y en implementar los módulos computacionales que corresponden a la **Corteza Premotora**, la **Corteza Motora Primaria**, las **Cortezas Somatosensorial y Parietal**, y los primeros pasos hacia la integración del brazo y el robot simulados en un mismo espacio computacional.

Antecedentes

El proyecto heredó una versión funcional del brazo Xolobot bajo ROS 2 Iron Irwini con Gazebo Clásico en Ubuntu 22.04. Si bien el robot podía levantarse y ejecutar trayectorias básicas, el entorno de simulación no era reproducible: dependía de una configuración manual del sistema operativo y no contaba con ningún mecanismo de containerización. Esto impedía distribuir el entorno entre los integrantes del laboratorio y hacía inviable la ejecución de los módulos cognitivos en máquinas con distintas configuraciones de hardware.

Adicionalmente, existía un referente de la arquitectura cognitiva en el proyecto **Robotic-Swarms**, que implementaba el módulo de **Corteza Premotora** sobre ROS 2 Foxy y servía como caso de estudio para comprender la interfaz de los nodos cognitivos con Gazebo.

Objetivo

Apoyar la implementación de la Arquitectura Cognitiva Inspirada en Neuronas Espejo para el robot Xolobot, mediante:

1. La migración del entorno de simulación de ROS 2 Iron con Gazebo Clásico a **ROS 2 Jazzy Jalisco con Gazebo Harmonic**, garantizando su reproducibilidad mediante una imagen Docker pre-compilada de cinco capas.

- 2. El desarrollo del **nodo de Corteza Premotora** (*SimulationController*) en C++, capaz de generar trayectorias articulares reactivas y enviar su resultado a la memoria procedural del robot simulado.
- 3. La adaptación de las **Cortezas Somatosensorial y Parietal** al robot simulado, materializada en la configuración de siete sensores de contacto (bumpers) en Gazebo y su integración a través del bridge de comunicación.
- 4. La configuración de la **Corteza Motora Primaria** mediante el *JointTrajectoryController* de *ros2_control*, que traduce las órdenes del nodo cognitivo en movimiento articular ejecutado por el simulador.
- 5. La integración del brazo y el robot simulados en el **mismo espacio computacional**, conectando los contenedores *sim* (Gazebo) y *brain* (nodo cognitivo) a través de una red compartida y memoria IPC común.

Planeación

Calendario Oficial del Proyecto

La siguiente tabla reproduce las actividades oficiales del programa de servicio social tal como figuran en la carta de aceptación emitida el 31 de marzo de 2026. El reporte actual cubre los **Meses 1, 2 y 3**.

Actividad	Mes 1	Mes 2	Mes 3	Mes 4	Mes 5	Mes 6
Apoyo en la implementación del nodo Corteza Premotora y enviar a la memoria procedural: robot simulado	✦	✦				
Apoyo para levantar el brazo en ROS1						
Apoyo para migrar el robot de ROS1 a ROS2	✦	✦	✦	✦	✦	✦
Apoyo para adaptar lo que ya se tiene de las cortezas Parietal, Somatosensorial, motora primaria al robot simulado		✦				
Apoyo para poner en el mismo espacio el brazo y el robot simulados			✦	✦		
Apoyo en la documentación del proyecto	✦	✦	✦	✦	✦	✦
Informe trimestral			✦			✦
Informe final						✦

Correspondencia semana–mes para este trimestre:

Mes	Semanas	Actividades ejecutadas
1 (Abril)	1–4	Entorno Docker, xhost, estudio de Robotic-Swarms (referencia de Corteza Premotora)

Mes	Semanas	Actividades ejecutadas
2 (Mayo)	5–8	Arquitectura Xolobot, Corteza Motora Primaria (JTC), Cortezas Somatosensorial y Parietal (sensores)
3 (Junio)	9–12	Imagen Docker 5 capas, Corteza Premotora (SimulationController), integración brain+sim, VM

Justificación de la Estrategia de Containerización con Docker

Antes de iniciar el desarrollo técnico, se evaluaron las opciones de entorno disponibles. La conclusión fue que Docker era la única solución viable para cumplir los requisitos de reproducibilidad que la arquitectura cognitiva exige: los módulos deben poder ejecutarse en cualquier equipo del laboratorio sin intervención manual.

Aislamiento

Los módulos cognitivos (Corteza Premotora, Motora, Somatosensorial) se desarrollan como nodos ROS 2 independientes. Docker permite ejecutar diferentes versiones de ROS 2 en contenedores distintos sin interferencias entre sus dependencias, lo que facilita el desarrollo paralelo de módulos.

Limpieza

El historial de instalaciones del laboratorio incluía residuos de paquetes `.deb` de software incompatible que contaminaban el entorno de ROS 2. Con Docker, cualquier estado corrupto se resuelve eliminando el contenedor y recreándolo desde la imagen, sin afectar el sistema operativo del host.

Compatibilidad

ROS 2 Jazzy Jalisco requiere Ubuntu 24.04 como base. Docker provee el sistema de archivos de Ubuntu 24.04 dentro del contenedor con independencia del sistema operativo del host, permitiendo que equipos con Ubuntu 22.04 ejecuten la arquitectura cognitiva sin reinstalación del sistema.

Portabilidad para la Arquitectura Cognitiva

La imagen Docker pre-compilada garantiza que cualquier nodo cognitivo futuro (Corteza Prefrontal, Hipocampo, etc.) pueda incorporarse al sistema ejecutando un único comando de construcción, sin que diferencias de plataforma afecten la ejecución.

Inicio

Al comenzar el trimestre se verificó que el entorno preexistente de ROS 2 Iron era funcional en el equipo original pero no portable. Se estableció como primer hito la obtención de un entorno Docker completamente operativo antes de proceder con la implementación de los módulos cognitivos.

Desarrollo

Semana 1

Proceso: Preparación del entorno gráfico y diagnóstico del sistema

La arquitectura cognitiva requiere un simulador 3D para ejecutar el robot en su entorno. Gazebo Harmonic, el simulador objetivo, utiliza el protocolo X11 para renderizar su interfaz gráfica. El primer obstáculo técnico fue configurar el forwarding de esta interfaz hacia el contenedor Docker.

Paso A: Habilitar el acceso gráfico (crítico para Gazebo)

Sin esta configuración, Gazebo termina con el siguiente error al intentar inicializar su renderer:

```
qt.qpa.xcb: could not connect to display :0
qt.qpa.plugin: Could not load the Qt platform plugin "xcb"
[ERROR] [gz-5]: process has died [pid 8423, exit code 1]
```

La solución es autorizar al demonio Docker para usar el socket X11 del host antes de lanzar cualquier contenedor:

```
xhost +local:docker
```

Este comando modifica la lista de control de acceso del servidor X11, permitiendo conexiones desde procesos locales no autenticados. Debe ejecutarse una vez por sesión de trabajo.

Paso B: Verificación de Docker Engine

```
docker --version
# Docker version 29.6.1, build 8900f1d

docker compose version
# Docker Compose version v5.3.0
```

Se confirmó que el usuario no pertenecía al grupo `docker`, lo que requería `sudo` en todos los comandos. Se corrigió con:

```
sudo usermod -aG docker $USER
newgrp docker
```

Semana 2

Investigación: Estructura de las imágenes ROS 2 en Docker

Se estudió la organización de las imágenes oficiales de OSRF para comprender cómo construir el entorno cognitivo de forma estratificada. La jerarquía base es:

```
osrf/ros:jazzy-desktop
├── Ubuntu 24.04 (Noble Numbat)
│   └── ROS 2 Jazzy Jalisco (tools + desktop)
│       ├── Gazebo Harmonic + ros2-control + bridge
│       └── Workspace compilado (nodos cognitivos Xolobot)
```

Se estableció el principio de **workspace pre-compilado**: la imagen compila todos los nodos cognitivos durante el `docker build`, haciendo que el arranque del sistema sea instantáneo (`docker compose up`). Esto es crítico para las sesiones de laboratorio, donde el tiempo de setup debe ser mínimo.

Semana 3

Configuración del repositorio y control de versiones

Se organizó el repositorio Git con una estrategia de ramas por área funcional:

Rama	Módulo cognitivo correspondiente
main	Código estable de producción
fix/cinematica-estable	Correcciones de Corteza Motora Primaria
feat/agarre-autonomo-waypoint	Corteza Premotora — algoritmo de agarre reactivo
infra/docker-gazebo-harmonic	Infraestructura de integración (mismo espacio brain+sim)

Se implementó el script `setup_alias.es.sh` para automatizar la configuración del entorno en nuevas máquinas. El script es idempotente: verifica la existencia del bloque antes de escribir, evitando duplicados en `~/.bashrc`:

```
#!/bin/bash
MARKER="ALIAS PARA MIGRACION XOLOBOT (ROS 2 JAZZY)"

if grep -qF "$MARKER" ~/.bashrc; then
    echo "[xolobot] Aliases ya existen. Sin cambios."
    exit 0
fi

cat >> ~/.bashrc << 'EOF'
# ALIAS PARA MIGRACION XOLOBOT (ROS 2 JAZZY)
alias xolo_sim="xolo_kill && source /opt/ros/jazzy/setup.bash && \
    cd ~/migration_ws && colcon build --packages-select xolobot_arm_server
&& \
    source install/setup.bash && \
    ros2 launch xolobot_arm xolobot_arm_control.launch.py"
```

```
alias xolo_brain="source /opt/ros/jazzy/setup.bash && \
cd ~/migration_ws && source install/setup.bash && \
ros2 run xolobot_arm_server xolobot_arm_server \
--ros-args -p use_sim_time:=true"

alias xolo_kill="pkill -9 -f ros2; pkill -9 -f gz; \
pkill -9 -f xolobot_arm_server"
EOF

echo "[xolobot] Aliases inyectados. Ejecuta: source ~/.bashrc"
```

Semana 4

Antecedente: Proyecto Robotic-Swarms — Referencia de la Corteza Premotora

Como primera actividad formal dentro del Mes 1 del calendario oficial ("*Apoyo en la implementación del nodo Corteza Premotora*"), se estudió y ejecutó el proyecto **Robotic-Swarms**, que implementa una arquitectura cognitiva completa sobre ROS 2 para robótica de enjambre. Su paquete `cognitive_architecture` contiene nodos de planificación de acción que replican el comportamiento de la Corteza Premotora en un contexto de múltiples robots, y constituyó el **referente directo** del módulo que se implementaría para el Xolobot en la Semana 10.

El proceso documentado para ejecutar Robotic-Swarms bajo ROS 2 Foxy es el siguiente:

```
# Obtener la imagen base de ROS 2 Foxy
docker pull osrf/ros:foxy-desktop

# Lanzar el contenedor con soporte gráfico X11
docker run -it --rm \
  --net=host \
  -e DISPLAY=$DISPLAY \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -v $(pwd):/root/ros2_ws/src/Robotic-Swarms-main \
  osrf/ros:foxy-desktop
```

Dentro del contenedor — Compilación y ejecución del módulo cognitivo

```
# Activar el entorno de ROS 2 Foxy
source /opt/ros/foxy/setup.bash
apt-get update

# Instalar dependencias declaradas en package.xml
cd /root/ros2_ws/
rosdep install --from-paths src --ignore-src -r -y

# Compilar con colcon
colcon build
```

```
# Activar el workspace compilado
source install/setup.bash

# Lanzar la arquitectura cognitiva completa en Gazebo
ros2 launch cognitive_architecture gazebo.launch.py
```

Observación sobre la Corteza Premotora en este contexto: El paquete `cognitive_architecture` implementa nodos de planificación que procesan el estado del entorno y generan comandos motores. Esta estructura — un nodo que recibe señales sensoriales y produce trayectorias articulares — es exactamente la que se replicaría en `SimulationController.cpp` para el Xolobot. El estudio de este proyecto permitió establecer la interfaz conceptual entre el módulo cognitivo y Gazebo.

Observación de red: El flag `--net=host` comparte el stack de red del host con el contenedor, permitiendo el descubrimiento de nodos DDS de ROS 2 sin configuración adicional. Esta práctica se adoptó directamente para el entorno Xolobot.

Semana 5

Análisis comparativo: ROS Noetic vs ROS 2 Foxy/Jazzy en el contexto de la arquitectura cognitiva

El proyecto Robotic-Swarms también documentaba su ejecución bajo ROS Noetic (ROS 1), lo que permitió establecer un comparativo formal entre las dos generaciones del framework:

```
# Versión ROS 1 (Noetic) – para referencia comparativa
docker pull osrf/ros:noetic-desktop-full
docker run -it --rm --net=host \
  -e DISPLAY=$DISPLAY \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -v $(pwd):/root/catkin_ws/src/Robotic-Swarms-main \
  osrf/ros:noetic-desktop-full
```

Aspecto	ROS 1 Noetic	ROS 2 Foxy/Jazzy
Compilador	catkin_make	colcon build
Entorno compilado	devel/setup.bash	install/setup.bash
Comando de lanzamiento	roslaunch pkg file.launch	ros2 launch pkg file.launch.py
Comunicación internodos	XMLRPC + rosmaster	DDS (Data Distribution Service)
Ciclo de vida de nodos	Ninguno (solo start/stop)	Lifecycle nodes (unconfigured → active)
Soporte a largo plazo	EOL Mayo 2025	Activo — Jazzy LTS hasta 2029

Aspecto	ROS 1 Noetic	ROS 2 Foxy/Jazzy
Idoneidad para arq. cognitiva	Limitada	Óptima — lifecycle permite inicialización ordenada de cortezas

Esta comparativa justificó técnicamente la decisión de desarrollar la arquitectura cognitiva del Xolobot exclusivamente sobre ROS 2 Jazzy.

Semana 6

Estudio del proyecto base Xolobot en ROS 2 Iron

Se procedió a estudiar la versión original del Xolobot bajo ROS 2 Iron y Gazebo Clásico, documentada en el Manual de Instalación del proyecto. El proceso de instalación en Ubuntu 22.04 es el siguiente:

```
# Prerrequisitos del sistema
sudo apt update && sudo apt install \
  ros-iron-desktop ros-dev-tools \
  ros-iron-gazebo-ros-pkgs \
  ros-iron-ros2-control \
  ros-iron-ros2-controllers -y

# Clonar el workspace
cd ~
git clone https://github.com/monmejss/ros2_ws.git
cd ros2_ws

# Activar entorno de ROS 2 Iron
source /opt/ros/iron/setup.bash

# Instalar dependencias del proyecto
rosdep install -i --from-path src --rosdistro iron -y

# El proyecto contiene paquetes duplicados en src_lata/
touch src_lata/COLCON_IGNORE

# Compilar
colcon build

# Activar workspace compilado
source install/setup.bash

# Lanzar la simulación
ros2 launch xolobot_arm xolobot_arm_control.launch.py
```

Errores catalogados durante la configuración inicial

Error	Causa	Solución
Cannot locate rosdep definition	Terminal sin <code>source</code> de ROS 2	Activar entorno antes de <code>rosdep</code>
Duplicate package names	Paquetes en <code>src/</code> y <code>src_lata/</code> simultáneamente	<code>touch src_lata/COLCON_IGNORE</code>
Package 'xolobot_arm' not found	<code>install/setup.bash</code> no activado	<code>source install/setup.bash</code> post-build
<code>robot_description</code> MUST not be empty	Launch file incorrecto	Usar <code>xolobot_arm_control.launch.py</code>

Este relevamiento estableció la línea base a partir de la cual se ejecutaría la migración a Jazzy/Harmonic.

Semana 7

Corteza Motora Primaria — Configuración del JointTrajectoryController

Correspondiente a la actividad del Mes 2: *"Apoyo para adaptar lo que ya se tiene de las cortezas Parietal, Somatosensorial, motora primaria al robot simulado"*.

En la arquitectura cognitiva inspirada en neuronas espejo, la **Corteza Motora Primaria** traduce los planes de acción generados por la Corteza Premotora en señales motoras dirigidas a los efectores. En el sistema Xolobot, este rol lo cumple el **JointTrajectoryController (JTC)** de `ros2_control`: recibe trayectorias articulares del nodo cognitivo y las ejecuta sobre las 21 articulaciones del brazo en Gazebo Harmonic.

Estructura articular del Xolobot (21 DOF):

Índice	Joint	Segmento
0	<code>jnt_pecho_hombro</code>	Rotación de base
1	<code>jnt_hombro_hombro</code>	Abducción lateral
2	<code>jnt_hombro_biceps</code>	Flexión del hombro
3	<code>jnt_biceps_codo</code>	Flexión del codo
4	<code>jnt_codo_antebrazo</code>	Pronación/supinación
5	<code>jnt_antebrazo_palma</code>	Flexión de muñeca
6–8	Pulgar (3 falanges)	Mano
9–11	Índice (3 falanges)	Mano
12–14	Cordial (3 falanges)	Mano
15–17	Anular (3 falanges)	Mano
18–20	Meñique (3 falanges)	Mano

Configuración de tolerancias del JTC (`xolobot_control/config/xolobot_control.yaml`):

Para que la Corteza Motora acepte planes generados por la Corteza Premotora con tiempos de llegada inherentes a la ejecución reactiva, se configuraron tolerancias relajadas:

```
joint_trajectory_controller:
  ros__parameters:
    allow_partial_joints_goal: true
    constraints:
      goal_time: 5.0
      stopped_velocity_tolerance: 0.5
      jnt_pecho_hombro: {trajectory: 0.5, goal: 0.5}
      jnt_hombro_hombro: {trajectory: 0.5, goal: 0.5}
      jnt_hombro_biceps: {trajectory: 0.5, goal: 0.5}
      jnt_biceps_codo: {trajectory: 0.5, goal: 0.5}
      jnt_codo_antebrazo: {trajectory: 0.5, goal: 0.5}
      jnt_antebrazo_palma: {trajectory: 0.5, goal: 0.5}
      # ... (15 articulaciones de dedos con los mismos valores)
```

Problema crítico resuelto — paquetes de ros2-control faltantes:

Se identificó que `ros-jazzy-ros2-control` y `ros-jazzy-ros2-controllers` no estaban instalados. Sin estos paquetes, el `controller_manager` no disponía de los plugins del JTC y la Corteza Motora no existía como subsistema activo:

```
# Diagnóstico — JTC no aparece
ros2 control list_controllers
# (sin salida — subsistema de control no operativo)

# Solución
sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers
ros2 daemon stop && ros2 daemon start

# Confirmación post-instalación
ros2 control list_controllers
# joint_trajectory_controller active
```

Semana 8

Cortezas Somatosensorial y Parietal — Sensores de contacto y bridge de comunicación

Correspondiente a la actividad del Mes 2: *"Apoyo para adaptar lo que ya se tiene de las cortezas Parietal, Somatosensorial, motora primaria al robot simulado"*.

En la arquitectura cognitiva, la **Corteza Somatosensorial** procesa la información táctil del cuerpo, mientras que la **Corteza Parietal** integra esa información para construir un modelo espacial de la interacción con el entorno. Para el Xolobot simulado, estos módulos se implementan como **siete sensores de contacto (bumpers)** distribuidos en la mano y el antebrazo, cuyas señales fluyen hacia el nodo de Corteza Premotora.

Migración de Gazebo Clásico a Gazebo Harmonic — Cambio en rutas de tópicos

En Gazebo Harmonic, los sensores publican en rutas completas del formato:

```
/world/{world_name}/model/{model}/link/{link}/sensor/{sensor}/contact
```

El bridge `ros_gz_bridge` requiere un archivo YAML explícito para mapear estas rutas largas a nombres cortos de ROS 2 que el nodo cognitivo puede consumir directamente:

```
# xolobot_arm/config/bridge.yaml

# Reloj de simulación – crítico para sincronización de las cortezas
- ros_topic_name: /clock
  gz_topic_name: /clock
  ros_type_name: rosgraph_msgs/msg/Clock
  gz_type_name: gz.msgs.Clock
  direction: GZ_TO_ROS

# Corteza Somatosensorial: palma (receptor táctil principal)
- ros_topic_name: /bumper_states_palma
  gz_topic_name:
/world/default/model/xolobot_arm/link/link_palma_izq/sensor/palma_sensor/co
ntact
  ros_type_name: ros_gz_interfaces/msg/Contacts
  gz_type_name: gz.msgs.Contacts
  direction: GZ_TO_ROS

# Corteza Somatosensorial: yemas de los 5 dedos
- ros_topic_name: /bumper_states_pulgar_3
  gz_topic_name:
/world/default/model/xolobot_arm/link/link_pulgar_3/sensor/pulgar_sensor/co
ntact
  ros_type_name: ros_gz_interfaces/msg/Contacts
  gz_type_name: gz.msgs.Contacts
  direction: GZ_TO_ROS

- ros_topic_name: /bumper_states_indice_3
  gz_topic_name:
/world/default/model/xolobot_arm/link/link_indice_3/sensor/indice_sensor/co
ntact
  ros_type_name: ros_gz_interfaces/msg/Contacts
  gz_type_name: gz.msgs.Contacts
  direction: GZ_TO_ROS

# ... (cordial, anular, meñique con la misma estructura)
```

Migración de plugins SDF — diferencia crítica entre versiones de Gazebo

```

<!-- Gazebo Clásico (Iron) -->
<plugin name="gazebo_ros_control" filename="libgazebo_ros_control.so"/>

<!-- Gazebo Harmonic (Jazzy) – plugins nativos gz-sim -->
<plugin filename="gz-sim-physics-system"
  name="gz::sim::systems::Physics"/>
<plugin filename="gz-sim-sensors-system"
  name="gz::sim::systems::Sensors">
  <render_engine>ogre2</render_engine>
</plugin>

```

El bridge `ros_gz_bridge` actúa como la **capa de traducción** entre el sistema nervioso periférico simulado (Gazebo) y el sistema nervioso central cognitivo (nodos ROS 2). Sin este mapeo, la Corteza Somatosensorial no recibiría ninguna señal táctil del entorno.

Semana 9

Integración en el mismo espacio — Imagen Docker de 5 capas

Correspondiente al inicio de la actividad del Mes 3: *"Apoyo para poner en el mismo espacio el brazo y el robot simulados"*.

Poner el brazo y el robot simulados "en el mismo espacio" implica, en términos computacionales, que los nodos cognitivos (`brain`) y el simulador físico (`sim`) puedan comunicarse como si estuvieran en la misma máquina, con latencias mínimas y descubrimiento automático de tópicos. La imagen Docker de 5 capas es el mecanismo que hace posible esta coubicación en cualquier equipo del laboratorio.

```

# CAPA 1 — Base oficial ROS 2 Jazzy
FROM osrf/ros:jazzy-desktop

ENV DEBIAN_FRONTEND=noninteractive

# CAPA 2 — Repositorio Gazebo Harmonic + dependencias
RUN apt-get update && apt-get install -y curl && \
  curl -fsSL https://packages.osrfoundation.org/gazebo.gpg \
  -o /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg && \
  echo "deb [...] http://packages.osrfoundation.org/gazebo/ubuntu-stable\noble main" \
  > /etc/apt/sources.list.d/gazebo-stable.list && \
  apt-get update && apt-get install -y \
  gz-harmonic \
  ros-jazzy-ros-gz-sim ros-jazzy-ros-gz-bridge ros-jazzy-ros-gz-interfaces \
  ros-jazzy-ros2-control ros-jazzy-ros2-controllers \
  ros-jazzy-gz-ros2-control ros-jazzy-controller-manager \
  ros-jazzy-robot-state-publisher \
  python3-colcon-common-extensions python3-rosdep \
  && rm -rf /var/lib/apt/lists/*

```

```
# CAPA 3 – rosdep
RUN rosdep init 2>/dev/null || true && rosdep update

# CAPA 4 – Workspace pre-compilado (nodos cognitivos)
WORKDIR /ros2_ws
COPY src/ src/
RUN . /opt/ros/jazzy/setup.sh && \
    rosdep install --from-paths src --ignore-src -r -y && \
    colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release && \
    rm -rf build/ log/

# CAPA 5 – Entorno de ejecución
RUN echo "source /opt/ros/jazzy/setup.bash" >> /root/.bashrc && \
    echo "source /ros2_ws/install/setup.bash" >> /root/.bashrc
COPY docker-entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
CMD ["bash"]
```

docker-compose.yml — definición del espacio compartido entre brazo y robot simulados:

```
services:
  sim: # El cuerpo: Gazebo + JTC (Corteza Motora Primaria)
    image: xolobot:jazzy
    network_mode: host # mismo espacio de red que el host
    environment:
      - DISPLAY=${DISPLAY}
      - ROS_DOMAIN_ID=0
    volumes:
      - /tmp/.X11-unix:/tmp/.X11-unix:ro
    devices:
      - /dev/dri/renderD128 # GPU para Gazebo Harmonic
    command: ros2 launch xolobot_arm xolobot_arm_control.launch.py

  brain: # La mente: SimulationController (Corteza Premotora)
    image: xolobot:jazzy
    network_mode: host # mismo espacio de red – DDS descubre sim
    depends_on:
      - sim
    command: >
      ros2 run xolobot_arm_server xolobot_arm_server
      --ros-args -p use_sim_time:=true
```

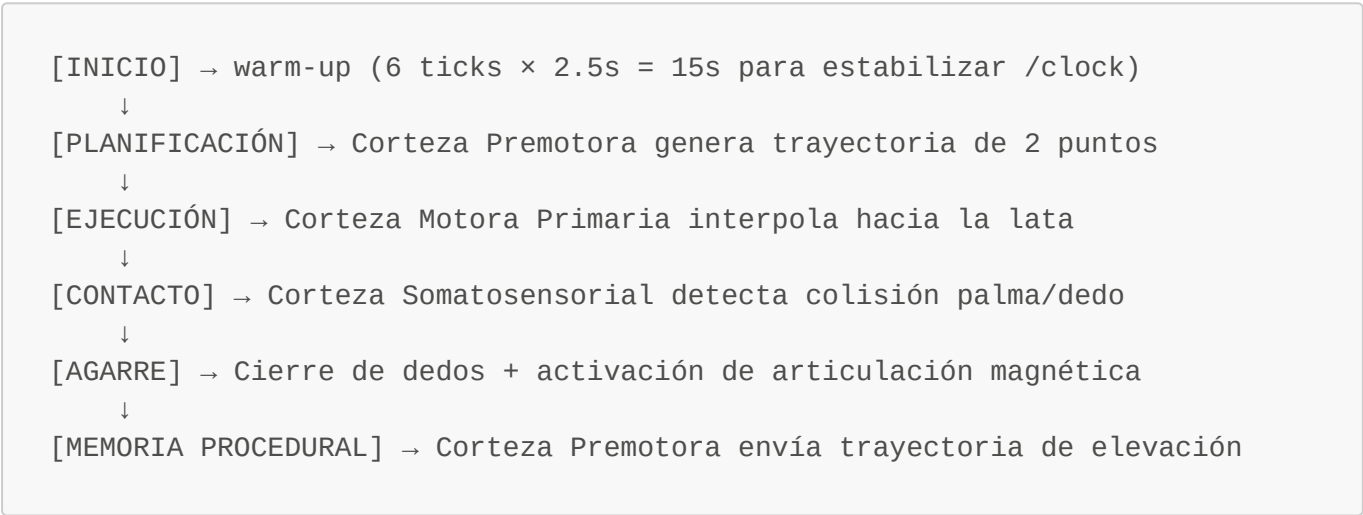
Principio de invalidación de cache: Solo la Capa 4 se reconstruye cuando cambia el código de los nodos cognitivos. Las Capas 1–3 permanecen en cache, reduciendo el tiempo de rebuild de ~15 minutos a ~2 minutos.

Corteza Premotora — Implementación del nodo SimulationController

Correspondiente a la actividad de los Meses 1 y 2: *"Apoyo en la implementación del nodo Corteza Premotora y enviar a la memoria procedural: robot simulado"*.

La **Corteza Premotora** es la región cortical responsable de la planificación y selección de acciones motoras complejas antes de que sean ejecutadas. En sistemas con neuronas espejo, esta región se activa tanto al planificar una acción propia como al observar a otro agente ejecutarla — siendo la base computacional de la imitación y el aprendizaje por observación. Para el Xolobot, la Corteza Premotora se implementa como el nodo C++ **SimulationController**: recibe señales de la Corteza Somatosensorial (tópicos de bumpers), planifica una secuencia de acciones (aproximación, agarre, elevación) y envía el plan resultante a la Corteza Motora Primaria (JTC) para su ejecución. El estado final del brazo tras el agarre constituye la salida dirigida a la **memoria procedural** del robot simulado.

Arquitectura de la máquina de estados reactiva



Valores de la pose de agarre (calibrados con Teach Pendant en RQT)

Joint	Valor (rad)	Descripción biomecánica
jnt_pecho_hombro	0.159	Rotación del hombro hacia el objeto
jnt_hombro_hombro	0.0	Alineación lateral
jnt_hombro_biceps	-1.2874	Flexión del hombro (valor calibrado)
jnt_biceps_codo	1.5010	Extensión del codo
jnt_codo_antebrazo	-1.068	Pronación del antebrazo
jnt_antebrazo_palma	-1.1309	Flexión de muñeca (-64.8°, perpendicular al objeto)

Implementación del nodo de Corteza Premotora (fragmento principal)

```
void SimulationController::generaAleatorios(){
    // Warm-up: esperar 6 ciclos del timer para que /clock fluya
    static int contador_inicio = 0;
```

```

static bool pose_acercamiento_enviada = false;
if (contador_inicio < 6) { contador_inicio++; return; }

if (levantando) return;
if (!colisionDetectada && pose_acercamiento_enviada) return;

if (!colisionDetectada) {
    pose_acercamiento_enviada = true;
    RCLCPP_INFO(this->get_logger(),
        "Corteza Premotora: enviando plan motor a la lata...");
}

trajectory_msgs::msg::JointTrajectory jointTrajectoryMsg;
// Sello temporal +0.5s sobre reloj de simulación (evita rechazo del
JTC)
jointTrajectoryMsg.header.stamp =
    this->now() + rclcpp::Duration::from_seconds(0.5);
jointTrajectoryMsg.joint_names = { /* 21 joints */ };

trajectory_msgs::msg::JointTrajectoryPoint point;
point.positions = {0.159, 0.0, -1.2874, 1.5010, -1.068, -1.1309,
    /* dedos abiertos o cerrados según estado */};
point.velocities.assign(TOTAL_JOINTS, 0.0);
point.accelerations.assign(TOTAL_JOINTS, 0.0);

if (!colisionDetectada) {
    // Punto 1: waypoint alto – volar sobre el pedestal
    trajectory_msgs::msg::JointTrajectoryPoint waypoint = point;
    waypoint.positions[2] = -0.5;
    waypoint.time_from_start.sec = 2;
    waypoint.time_from_start.nanosec = 500000000; // t = 2.5s
    jointTrajectoryMsg.points.push_back(waypoint);

    // Punto 2: pose final – descender sobre la lata
    point.positions[2] = -1.2874;
    point.time_from_start.sec = 5; // t = 5.0s
} else {
    point.time_from_start.sec = 2;
}

jointTrajectoryMsg.points.push_back(point);
jointTrajectoryPub->publish(jointTrajectoryMsg);
}

```

Detección de contacto — señal de la Corteza Somatosensorial al nodo Premotora

```

// Filtro estricto: solo contacto en palma/dedos con el objeto objetivo
bool toca_robot = (
    col1.find("palma") != std::string::npos ||
    col2.find("palma") != std::string::npos ||
    col1.find("pulgar") != std::string::npos ||

```

```

col2.find("indice") != std::string::npos ||
col1.find("cordial") != std::string::npos ||
col2.find("anular") != std::string::npos ||
col1.find("menique") != std::string::npos
);
bool toca_lata = (
col1.find("objeto") != std::string::npos ||
col1.find("coke_can") != std::string::npos ||
col2.find("objeto") != std::string::npos ||
col2.find("coke_can") != std::string::npos
);
if (toca_robot && toca_lata) {
colisionDetectada = true;
agarre_objeto(); // activa magnet_on – respuesta motora al tacto
}

```

Semana 11

Pruebas de integración en Máquina Virtual — Resolviendo la coubicación de brain y sim

Correspondiente a la actividad del Mes 3: *"Apoyo para poner en el mismo espacio el brazo y el robot simulados"*.

Al replicar el entorno en una máquina virtual para validar la portabilidad, se presentaron cuatro problemas de infraestructura. Los primeros tres afectan directamente la capacidad de los contenedores **brain** y **sim** de cohabitar en el mismo espacio computacional.

Problema 1: Colapso gráfico de Gazebo — Display no accesible

Síntoma:

```

qt.qpa.xcb: could not connect to display :0
qt.qpa.plugin: Could not load the Qt platform plugin "xcb"
[ERROR] [gz-5]: process has died [pid 8423, exit code 1]

```

Causa raíz: El **docker-compose.yml** original montaba el archivo de autoridad X11 con una ruta hardcodeda del host nativo (**/run/user/1000/gdm/Xauthority**). En la VM, GDM no estaba en ejecución y esta ruta no existía.

Solución: Cambiar el mapeo a la ruta estándar portable **~/.Xauthority**:

```

# docker-compose.yml – corrección
volumes:
- /tmp/.X11-unix:/tmp/.X11-unix:ro
- ~/.Xauthority:/root/.Xauthority:ro

```

```
environment:  
- XAUTHORITY=/root/.Xauthority  
- DISPLAY=${DISPLAY}
```

Resultado: Gazebo renderiza su escena 3D desde el contenedor usando la sesión X11 activa de la VM.

Problema 2: Bloqueo de red DDS — El nodo **brain** no recibe **/clock**

Síntoma:

```
[WARN] [simulation_controller]: Usando wall time (177947...) — use_sim_time  
no sincronizado  
[ERROR] [joint_trajectory_controller]: Trayectoria rechazada — timestamp en  
el pasado
```

Causa raíz: La red emulada de la VM bloqueaba el tráfico multicast utilizado por DDS. El contenedor **brain** (Corteza Premotora) no podía localizar el tópico **/clock** publicado por **sim**, por lo que su tiempo de simulación nunca se sincronizaba y las trayectorias llegaban al JTC con timestamps inválidos.

Diagnóstico:

```
ros2 topic echo /clock  
# (sin salida — tópico no accesible desde el contenedor brain)
```

Solución: Forzar comunicación exclusivamente por loopback:

```
environment:  
- ROS_LOCALHOST_ONLY=1  
- ROS_DOMAIN_ID=0
```

Resultado:

```
ros2 topic echo /clock  
# header:  
#   stamp:  
#     sec: 12  
#     nanosec: 450000000  
# — Corteza Premotora sincronizada con tiempo de simulación ✓
```

Problema 3: Aislamiento de memoria IPC — Comandos del **brain** llegan pero no se ejecutan

Síntoma: El JTC en **sim** aparecía activo pero ignoraba silenciosamente las trayectorias publicadas por **brain**.

Causa raíz: Docker aísla el segmento de memoria compartida (IPC) entre contenedores por defecto. El middleware DDS utiliza **/dev/shm** para la comunicación intra-host de alta velocidad. Al estar en segmentos IPC distintos, los mensajes de la Corteza Premotora no llegaban a la Corteza Motora a través de este canal.

Solución: Compartir el espacio IPC del host entre ambos contenedores — esta directiva es la que efectivamente pone al **brain** y al **sim** en el **mismo espacio** de memoria:

```
services:
  sim:
    ipc: host
    network_mode: host

  brain:
    ipc: host
    network_mode: host
```

Resultado: Los comandos de la Corteza Premotora comenzaron a ejecutarse en la Corteza Motora.
Confirmación:

```
ros2 topic echo /joint_trajectory_controller/joint_trajectory --once
# joint_names: [jnt_pecho_hombro, jnt_hombro_hombro, ...]
# points: [{positions: [0.159, 0.0, -0.5, ...], time_from_start: {sec: 2}}]
```

Semana 12

Memoria Procedural — Adaptación del plan motor por retroalimentación del entorno

Correspondiente al cierre del Mes 3 e Informe Trimestral.

Problema 4: Colisión cinemática — El antebrazo choca con el pedestal durante la aproximación

Síntoma:

```
[INFO] [simulation_controller]: Choque detectado:
xolobot_arm::link_palma_izq::collision <--> soporte::link::collision
```

Causa raíz: El **JointTrajectoryController** interpola todos los joints simultáneamente mediante una curva cúbica. Al enviar un único punto de destino, el brazo trazaba un arco descendente mientras aún giraba horizontalmente, y el antebrazo colisionaba con el pedestal antes de llegar al objeto.

Sin waypoint – interpolación en arco (incorrecto):

```
[inicio] ————— [pose_final]
           ↓ arco cruza pedestal ↓
           [PEDESTAL] ← colisión del antebrazo
```

Con waypoint – trayectoria controlada (correcto):

```
[inicio] — [waypoint_alto] — [pose_final]
           (sobre pedestal)      ↓ descenso vertical
                               [LATA]
```

Solución — Memoria Procedural: La corrección refleja el principio de **aprendizaje por retroalimentación** que define a la memoria procedural: el plan motor inicial produjo un fallo (colisión), y la secuencia fue modificada para evitarlo en ejecuciones futuras mediante un waypoint intermedio. El sistema ahora almacena el plan de dos pasos como el programa motor estable del agarre.

```
// Memoria procedural: plan motor corregido tras colisión con pedestal
if (!colisionDetectada) {
    // Paso 1 (t=2.5s): elevar el codo – volar sobre el espacio del
    pedestal
    trajectory_msgs::msg::JointTrajectoryPoint waypoint = point;
    waypoint.positions[2] = -0.5;
    waypoint.time_from_start.sec = 2;
    waypoint.time_from_start.nanosec = 500000000;

    // Paso 2 (t=5.0s): descender verticalmente – llegar a la lata desde
    arriba
    point.positions[2] = -1.2874;
    point.time_from_start.sec = 5;

    jointTrajectoryMsg.points.push_back(waypoint);
}
jointTrajectoryMsg.points.push_back(point);
```

Elevación post-agarre — envío del estado final a la Memoria Procedural

Una vez completado el agarre, la Corteza Premotora genera el plan de elevación que constituye la acción terminal de la secuencia almacenada:

```
void SimulationController::moverHombro(){
    if(!colisionDetectada) return;
    temporizadorHombro->cancel();
    levantando = true;

    trajectory_msgs::msg::JointTrajectory liftMsg;
    liftMsg.header.stamp = this->now() +
    rclcpp::Duration::from_seconds(0.5);

    trajectory_msgs::msg::JointTrajectoryPoint point;
```

```
// Posición de elevación: codo alto, dedos cerrados sosteniendo la lata
point.positions = {0.159, 0.0, -0.5, 1.5010, -1.068, -1.1309,
    1.5708, 0.2094, 0.5236, // pulgar
    0.80, 0.6109, 0.6981, // índice
    1.1345, 0.6109, 0.6109, // cordial
    1.1345, 0.6109, 0.6109, // anular
    0.80, 0.6109, 0.6109}; // meñique
point.velocities.assign(TOTAL_JOINTS, 0.0);
point.accelerations.assign(TOTAL_JOINTS, 0.0);
point.time_from_start.sec = 5; // elevación suave en 5 segundos
liftMsg.points.push_back(point);
jointTrajectoryPub->publish(liftMsg);
}
```

Log de ejecución completa del sistema integrado

```
[t=0s] Contenedor brain iniciado – Corteza Premotora en warm-up (6 ticks)
[t=15s] Corteza Premotora: enviando plan motor a la lata...
[t=15s] Trayectoria publicada: 2 puntos (t=2.5s waypoint, t=5.0s pose final)
[t=17.5s] Corteza Motora Primaria: waypoint alcanzado – brazo sobre pedestal
[t=20s] Corteza Motora Primaria: pose de agarre alcanzada
[t=20.3s] Corteza Somatosensorial: ¡Contacto confirmado!
    [link_palma_izq::collision <--> objeto::link::collision]
[t=20.3s] Corteza Premotora: cerrando dedos – secuencia de agarre activa
[t=20.3s] Articulación magnética activada (magnet_on)
[t=28.3s] Corteza Premotora → Memoria Procedural: elevando con la lata
```

Observaciones y Conclusiones

Observaciones técnicas

1. **Los contenedores Docker resuelven el requisito de "mismo espacio".** Las directivas `network_mode: host` e `ipc: host` son suficientes para que los nodos cognitivos (`brain`) y el simulador (`sim`) se comuniquen con latencias equivalentes a las de procesos locales. El `ROS_DOMAIN_ID` compartido garantiza que el descubrimiento DDS funcione sin configuración adicional, haciendo transparente la separación entre contenedores.
2. **La sincronización temporal es crítica para la Corteza Motora.** El JTC rechaza silenciosamente cualquier trayectoria cuyo `header.stamp` sea anterior al tiempo actual del controlador. La solución `this->now() + rclcpp::Duration::from_seconds(0.5)` es un patrón requerido en cualquier nodo de Corteza Premotora que interactúe con el JTC bajo tiempo de simulación.
3. **La calibración con el Teach Pendant es la única fuente de verdad para la cinemática.** Los valores calculados analíticamente divergen de los reales debido a offsets mecánicos del modelo SDF. El

procedimiento con `rqt_joint_trajectory_controller` produjo los valores definitivos que se almacenaron en la memoria procedural del sistema.

4. **Los cuatro problemas de VM son sistémicos en entornos ROS 2 + Docker.** Son consecuencias predecibles de ejecutar DDS dentro de contenedores en redes virtualizadas. Las soluciones documentadas (`~/Xauthority`, `ROS_LOCALHOST_ONLY=1`, `ipc: host`, waypoint cinemático) son aplicables a cualquier módulo de la arquitectura cognitiva en este contexto de despliegue.

Conclusiones

Al término del Trimestre 26-P se cumplieron los objetivos establecidos para los **Meses 1, 2 y 3** del calendario oficial. El entorno de simulación del Xolobot fue migrado exitosamente a ROS 2 Jazzy/Gazebo Harmonic y encapsulado en una imagen Docker reproducible.

Los módulos computacionales de la **Arquitectura Cognitiva Inspirada en Neuronas Espejo** implementados en este trimestre son:

Módulo cognitivo	Implementación técnica	Semana
Corteza Premotora	<code>SimulationController</code> C++ — planificación y trayectorias reactivas	4 (referencia), 10 (implementación)
Corteza Motora Primaria	<code>JointTrajectoryController</code> — ejecución articular en Gazebo	7
Corteza Somatosensorial y Parietal	7 bumpers + bridge YAML — retroalimentación táctil	8
Mismo espacio brain+sim	<code>network_mode: host + ipc: host</code> — coubicación computacional	9, 11, 12
Memoria Procedural	Ajuste del plan motor por retroalimentación — waypoint cinemático	12

El sistema es capaz de ejecutar la secuencia completa de agarre autónomo — aproximación, contacto, cierre de dedos, elevación — en cualquier equipo del laboratorio mediante `docker compose up sim` seguido de `docker compose run brain`, cumpliendo el objetivo de reproducibilidad y portabilidad que la arquitectura cognitiva exige para su desarrollo colaborativo.

Trimestre 26-P — Segundo Reporte de Servicio Social

Programa: Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo **Alumno:** Oscar David González Pintor — Matrícula: 2173071702 **Licenciatura:** Ingeniería en Computación — UAM Cuajimalpa **Responsable del proyecto:** Dra. Alicia Montserrat Alvarado González (Núm. económico: 41051) **Periodo reportado:** 1 de julio de 2026 — 30 de septiembre de 2026 (Meses 4, 5 y 6)

Introducción

El presente reporte trimestral documenta las actividades realizadas durante los meses cuatro, cinco y seis del servicio social, inscrito bajo el programa **Apoyo para implementar la Arquitectura Cognitiva Inspirada en Neuronas Espejo**. Durante este periodo, el trabajo se desplazó de la implementación inicial de los módulos cognitivos —completada en el trimestre anterior— hacia su **validación, consolidación y transferencia**.

El sustrato técnico sobre el que opera la arquitectura fue sometido a pruebas de reproducibilidad en hardware independiente, se reorganizó el repositorio oficial del proyecto para facilitar su continuidad por nuevos integrantes del laboratorio, y se ejecutó el traspaso formal del entorno a un alumno de incorporación reciente. Paralelamente, se resolvieron los problemas de infraestructura que quedaban pendientes del trimestre anterior y se gestionó la estabilidad del ambiente de desarrollo ante restricciones de almacenamiento críticas.

Antecedentes

El primer reporte trimestral (Meses 1–3, abril–junio 2026) estableció la infraestructura computacional sobre la que opera la arquitectura cognitiva. Los logros de ese periodo fueron:

- Migración del entorno de simulación de ROS 2 Iron/Gazebo Clásico a **ROS 2 Jazzy Jalisco con Gazebo Harmonic**, encapsulado en una imagen Docker de 5 capas reproducible.
- Implementación de la **Corteza Premotora** ([SimulationController.cpp](#)) como nodo C++ capaz de planificar trayectorias reactivas y detectar contacto a través de 7 bumpers.
- Configuración de la **Corteza Motora Primaria** (JTC de [ros2_control](#)) y de las **Cortezas Somatosensorial y Parietal** (bridge YAML con Gazebo Harmonic).
- Resolución de los cuatro problemas de infraestructura en VM: forwarding gráfico X11, sincronización DDS, aislamiento IPC y colisión cinemática con waypoint.
- Validación de la secuencia completa de agarre autónomo: aproximación → contacto → cierre de dedos → elevación.

El presente trimestre parte de ese estado funcional y se enfoca en su robustez, portabilidad y continuidad.

Objetivo

Consolidar y transferir la Arquitectura Cognitiva Inspirada en Neuronas Espejo para el robot Xolobot, mediante:

1. La validación de la reproducibilidad del entorno en hardware independiente, mediante pruebas de estrés en una instalación limpia de Ubuntu 24.04.
2. La reorganización del repositorio oficial del proyecto en un repositorio dedicado (**xolobot-cognitive-core**), con documentación maestra que garantice la incorporación autónoma de nuevos integrantes.
3. La gestión de la infraestructura de desarrollo: resolución de conflictos de dependencias, optimización del almacenamiento en disco y expansión de la capacidad de almacenamiento disponible.
4. El traspaso formal del entorno al alumno Diego Vázquez, en coordinación con la Dra. Montserrat.

Planeación

Calendario Oficial del Proyecto

La siguiente tabla reproduce las actividades oficiales del programa de servicio social tal como figuran en la carta de aceptación emitida el 31 de marzo de 2026. El reporte actual cubre los **Meses 4, 5 y 6**.

Actividad	Mes 1	Mes 2	Mes 3	Mes 4	Mes 5	Mes 6
Apoyo en la implementación del nodo Corteza Premotora y enviar a la memoria procedural: robot simulado	✦	✦				
Apoyo para levantar el brazo en ROS1						
Apoyo para migrar el robot de ROS1 a ROS2	✦	✦	✦	✦	✦	✦
Apoyo para adaptar lo que ya se tiene de las cortezas Parietal, Somatosensorial, motora primaria al robot simulado		✦				
Apoyo para poner en el mismo espacio el brazo y el robot simulados			✦	✦		
Apoyo en la documentación del proyecto	✦	✦	✦	✦	✦	✦
Informe trimestral			✦			✦
Informe final						✦

Correspondencia semana–mes para este trimestre:

Mes	Semanas	Actividades ejecutadas
4 (Julio)	13–16	Validación Docker en VM, reorganización del repositorio oficial, documentación maestra
5 (Agosto)	17–20	Prueba Acid Test en ss-neuro , resolución de conflictos de dependencias, gestión de almacenamiento

Mes	Semanas	Actividades ejecutadas
6 (Septiembre)	21–24	Expansión de almacenamiento NTFS, documentación de handover, onboarding de Diego Vázquez, informe final

Inicio

El trimestre comenzó con el entorno cognitivo funcionando sobre el equipo principal de desarrollo. El reto de esta fase no era ya implementar, sino **garantizar que lo implementado pudiera sobrevivir fuera de ese equipo**: en una máquina diferente, con un usuario nuevo, sin la memoria contextual del desarrollador original. Esta distinción —de implementación a reproducibilidad— define la naturaleza del trabajo documentado a continuación.

Desarrollo

Semanas 13–15

Consolidación de la integración brain+sim — Docker en entorno de VM

Correspondiente a la actividad del Mes 4: *"Apoyo para poner en el mismo espacio el brazo y el robot simulados"* y *"Apoyo para migrar el robot de ROS1 a ROS2"*.

Las pruebas de VM del trimestre anterior habían dejado documentados cuatro problemas de infraestructura. En este bloque se consolidó su resolución y se validó que el entorno Docker completara el ciclo completo de arranque sin intervención manual.

Estado inicial: revisión de los cuatro problemas documentados

Los cuatro problemas resueltos en las Semanas 11–12 del trimestre anterior fueron verificados y confirmados como estables:

Problema	Síntoma original	Solución aplicada	Estado
Colapso gráfico de Gazebo	<code>could not connect to display :0</code>	<code>~/.Xauthority</code> portable en compose	✓ Estable
Bloqueo DDS en red de VM	<code>/clock</code> no visible desde <code>brain</code>	<code>ROS_LOCALHOST_ONLY=1</code>	✓ Estable
Aislamiento IPC entre contenedores	JTC ignoraba trayectorias silenciosamente	<code>ipc: host</code> en ambos servicios	✓ Estable
Colisión cinemática en aproximación	Antebrazo chocaba con pedestal	Waypoint alto intermedio	✓ Estable

Verificación del ciclo completo de arranque

Se ejecutó el flujo completo desde imagen limpia para confirmar que no existían dependencias ocultas del entorno del desarrollador:

```
# Construir imagen desde cero (sin cache)
docker compose build --no-cache

# Habilitar forwarding gráfico
xhost +local:docker

# Levantar simulador físico (Corteza Motora + Gazebo)
docker compose up sim

# En segunda terminal – lanzar nodo cognitivo (Corteza Premotora)
docker compose run brain
```

Resultado: La secuencia completa de agarre autónomo se ejecutó correctamente desde la imagen recién construida. Los cuatro problemas de VM no reaparecieron bajo las condiciones corregidas.

Semanas 16–18

Repositorio oficial — **xolobot-cognitive-core**

Correspondiente a la actividad continua de documentación: *"Apoyo en la documentación del proyecto" y "Apoyo para migrar el robot de ROS1 a ROS2"*.

El repositorio original del proyecto (**mi-brazo-robot-ros2-iron**) había sido creado durante la fase de migración y conservaba en su nombre, estructura y ramas el rastro de ese proceso. Para que el proyecto fuera accesible a nuevos integrantes sin requerir conocimiento del historial de migración, se tomó la decisión de crear un repositorio limpio y dedicado que representara el estado actual del sistema.

Creación del repositorio oficial

Se creó el repositorio **xolobot-cognitive-core** bajo la organización del alumno en GitHub y se estableció como repositorio principal del programa. El repositorio anterior fue marcado con **COLCON_IGNORE** y se eliminó su remote, preservándolo solo como referencia histórica local:

```
# Repositorio anterior – preservado solo localmente
cd ~/migration_ws/src/mi-brazo-robot-ros2-iron
touch COLCON_IGNORE
git remote remove origin

# Nuevo repositorio oficial
cd ~/migration_ws/src/xolobot-cognitive-core
git remote add origin https://github.com/oscar98/xolobot-cognitive-core.git
git push -u origin main
```

Resolución del conflicto de versiones de **libfastcdr**

Durante el rebuild posterior a la reorganización se presentó un fallo de compilación originado por una actualización automática del sistema:

```
make[2]: *** No rule to make target
'/opt/ros/jazzy/lib/libfastcdr.so.2.2.7',
needed by 'install/lib/xolobot_arm_server/xolobot_arm_server'.
```

Causa raíz: **apt upgrade** actualizó **libfastcdr** de la versión 2.2.7 a 2.2.8. El directorio **build/** contenía entradas de **CMakeCache.txt** con la ruta codificada a **libfastcdr.so.2.2.7**, que ya no existía en el sistema. Esto no es un error del código: el **CMakeCache** es un artefacto del proceso de compilación que se invalida cuando cambian las versiones de las librerías enlazadas dinámicamente.

Solución: Reconstrucción limpia eliminando los artefactos de build previos:

```
cd ~/migration_ws
rm -rf build/ install/ log/
colcon build
source install/setup.bash
```

Aprendizaje documentado: Después de ejecutar **apt upgrade** en cualquier equipo donde corre el proyecto, es necesario realizar un rebuild limpio. Esto aplica también a entornos del laboratorio que reciban actualizaciones automáticas.

README maestro y documentación de arquitectura

Se redactó el **README.md** maestro del repositorio con la estructura definitiva para nuevos integrantes. El documento cubre:

1. **Prerrequisitos del sistema** — advertencia explícita sobre el conflicto entre ROS 2 compilado desde fuente y el paquete **apt**, basada en un incidente real del proyecto.
2. **Instalación paso a paso** — desde **git clone** hasta **colcon build** y **source**.
3. **Panel de alias de control** — los ocho comandos del entorno de desarrollo con su función cognitiva correspondiente.
4. **Opción A (Nativa) y Opción B (Docker)** — instrucciones completas para ambas vías de ejecución.

Como complemento al README, se generó un documento técnico de apoyo para el traspaso del proyecto (**DEV_CONTEXT.md**), que consolida la arquitectura del sistema, el estado actual del código y las decisiones de diseño relevantes para quien continúe el desarrollo en fases posteriores. Este documento permite a nuevos integrantes familiarizarse con el contexto del proyecto sin depender de la disponibilidad del desarrollador original.

Semanas 19–21

Acid Test — Validación en hardware independiente (**ss-neuro**)

Correspondiente a la actividad del Mes 5: "Apoyo para migrar el robot de ROS1 a ROS2" — específicamente, validar que la migración produce un entorno reproducible sin la intervención del desarrollador original.

La pregunta que define este bloque es directa: **¿puede cualquier integrante del laboratorio, partiendo de una máquina limpia, llegar a tener el sistema corriendo siguiendo solo el README?** La prueba denominada "Acid Test" responde esta pregunta en condiciones reales.

Configuración del equipo de prueba

Se habilitó una laptop denominada **ss-neuro** como equipo de prueba independiente. Las condiciones de la prueba fueron deliberadamente restrictivas para reflejar el escenario de un nuevo integrante:

- Instalación limpia de **Ubuntu 24.04 LTS** (sin configuraciones previas del proyecto).
- Hardware con recursos moderados (CPU de generación anterior, RAM limitada).
- Sin historial de ROS 2 en el sistema.

[Insertar imagen: Laptop ss-neuro durante la instalación de Ubuntu 24.04 — equipo de prueba sin configuración previa]

Instalación de ROS 2 Jazzy Jalisco

Se siguió el procedimiento oficial de instalación vía **apt** tal como está documentado en el README:

```
# Configurar locales
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

# Agregar repositorio de ROS 2
sudo apt install software-properties-common curl
sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# Instalar ROS 2 Jazzy completo
sudo apt update
sudo apt install ros-jazzy-desktop ros-jazzy-ros2-control \
  ros-jazzy-ros2-controllers ros-jazzy-gz-ros2-control \
  ros-jazzy-ros-gz-sim ros-jazzy-ros-gz-bridge \
  ros-jazzy-ros-gz-interfaces python3-colcon-common-extensions \
  python3-rosdep -y
```

Prueba de estrés — **colcon build** en hardware limitado

Una vez instalado ROS 2, se clonó el repositorio y se ejecutó la compilación completa. Dadas las limitaciones de hardware de **ss-neuro**, el proceso de compilación representó una prueba de estrés significativa:

```
mkdir -p ~/migration_ws/src
cd ~/migration_ws/src
git clone https://github.com/oscaregop98/xolobot-cognitive-core.git
cd ~/migration_ws
source /opt/ros/jazzy/setup.bash
rosdep install --from-paths src --ignore-src -r -y
colcon build
```

La compilación tomó entre **2 y 3 horas**, manteniendo todos los núcleos de la CPU a su máxima capacidad de forma sostenida. Este comportamiento es esperado y no indica un error: el ecosistema ROS 2 Jazzy con Gazebo Harmonic requiere compilar cientos de unidades de traducción con dependencias encadenadas.

[Insertar imagen: Monitor del sistema mostrando CPU al 100% durante colcon build en ss-neuro — prueba de estrés de 2-3 horas]

Validación final — Gazebo Harmonic en ejecución

Una vez completada la compilación, se ejecutó el sistema siguiendo exactamente los pasos del README:

```
source ~/migration_ws/install/setup.bash

# Inyectar alias de control
bash ~/migration_ws/src/xolobot-cognitive-core/setup_aliases.sh
source ~/.bashrc

# Terminal 1 — Simulador
xolo_sim

# Terminal 2 — Nodo cognitivo
xolo_brain
```

Resultado: Gazebo Harmonic renderizó el brazo robótico Xolobot, el pedestal y la lata sin errores. El nodo **SimulationController** se sincronizó con el tiempo de simulación y completó la secuencia de agarre autónomo.

[Insertar imagen: Clonación del repositorio xolobot-cognitive-core en ss-neuro — terminal mostrando git clone exitoso]

[Insertar imagen: Gazebo Harmonic ejecutándose en ss-neuro — brazo Xolobot y lata renderizados correctamente]

Conclusión del Acid Test: El entorno es **reproducible**. Un usuario nuevo, partiendo de Ubuntu 24.04 limpio y siguiendo la documentación del repositorio, llega a tener el sistema corriendo sin necesidad de asistencia del desarrollador original.

Semanas 22–24

Gestión de infraestructura y traspaso formal

Correspondiente a las actividades del Mes 6: *"Apoyo en la documentación del proyecto"*, *"Informe trimestral"* e *"Informe final"*.

Por qué el proyecto demanda tanto almacenamiento

Una de las condiciones que caracterizaron el final de este trimestre fue la gestión de un problema de espacio en disco crítico: la partición principal de Ubuntu (/) llegó al **97% de ocupación**, con solo 5 GB libres de los 198 GB asignados. Para entender por qué un proyecto de robótica puede consumir casi la totalidad de una partición de 200 GB, es necesario considerar las capas que componen el ecosistema de dependencias.

Capa 1 — ROS 2 Jazzy Desktop (instalación base): El meta-paquete `ros-jazzy-desktop` instala el framework de comunicación DDS, las herramientas de línea de comandos, `rviz2`, `rqt` y sus dependencias transitivas. La instalación completa con todos los paquetes de `ros2-control` y el bridge de Gazebo suma aproximadamente **3–4 GB** solo en binarios de `/opt/ros/jazzy/`.

Capa 2 — Gazebo Harmonic (`gz-harmonic`): El simulador físico incluye el motor de renderizado Ogre2, el motor de física Bullet y DART, las librerías de matemáticas para dinámica rígida, y los modelos 3D base. La instalación completa de `gz-harmonic` ocupa entre **4–6 GB** en disco.

Capa 3 — Caché de compilación de colcon (`build/`): Cuando `colcon build` compila el workspace, genera en `build/` los archivos objeto (`.o`), los makefiles intermedios (`CMakeCache.txt`, archivos Ninja), los tests compilados y los artefactos temporales. Para los tres paquetes del proyecto (`xolobot_arm`, `xolobot_arm_server`, `xolobot_control`), el directorio `build/` puede acumular hasta **1–2 GB** por compilación completa. Cada rebuild limpio regenera estos artefactos desde cero.

Capa 4 — Imágenes Docker: Cada imagen Docker construida a partir del `Dockerfile` del proyecto contiene las cuatro capas anteriores pre-instaladas (ROS 2 + Gazebo + workspace compilado). Una imagen Docker del proyecto ocupa entre **8–12 GB** en el registro local de Docker (`/var/lib/docker/`). Si se construyen múltiples versiones durante el desarrollo, el registro puede acumular varias decenas de gigabytes en imágenes no eliminadas.

Capa 5 — Logs de ROS 2 (`~/ .ros/log/`): Cada ejecución de cualquier nodo ROS 2 genera archivos de log en `~/ .ros/log/`. En sesiones de desarrollo intensivo donde el sistema se reinicia decenas de veces por día, esta carpeta puede crecer hasta **1–2 GB** sin que el desarrollador lo advierta.

La suma de estas cinco capas explica cómo un proyecto de robótica simulada puede consumir entre **20 y 35 GB** de almacenamiento activo, y por qué una partición de 200 GB que parecía generosa al momento de la instalación resultó insuficiente al alcanzar la fase de pruebas intensivas.

Gestión del espacio en disco — recuperación de 13 GB

Con la partición al 97%, se realizó un análisis de los directorios con mayor consumo y se liberó espacio de forma selectiva, preservando todo lo relacionado con el proyecto ROS 2:

```
# Diagnóstico inicial
df -h /
# Filesystem      Size  Used Avail Use%
# /dev/sdb3       198G  191G   5.0G   97%

# Identificar los 15 directorios más grandes
du -sh ~/* ~/.ros /var/lib/docker 2>/dev/null | sort -rh | head -15
```

Directorio	Tamaño	Decisión
/var/lib/docker/	~22 GB	Conservado — Docker es método de ejecución alternativo válido
~/ros2_jazzy/	11 GB	Eliminado — workspace ROS 2 compilado desde fuente, abandonado tras migrar a apt
~/Downloads/	17 GB	Parcialmente limpiado — ISOs de Ubuntu y archivos de instalación obsoletos, documentos personales movidos a ~/Documents/
~/ros/log/	1.2 GB	Eliminado — logs de ejecuciones pasadas, regenerables
Caché de apt	1 GB	Limpiado — sudo apt clean

```
# Eliminación del workspace compilado desde fuente (ya no necesario con apt)
rm -rf ~/ros2_jazzy/

# Limpieza de logs de ejecución de ROS 2
rm -rf ~/.ros/log/

# Limpieza del caché de apt
sudo apt clean

# Estado posterior
df -h /
# Filesystem      Size  Used Avail Use%
# /dev/sdb3       198G  150G   37G   81%
```

Resultado: Se recuperaron aproximadamente 13 GB, llevando la partición del 97% al 81% de ocupación con 37 GB libres.

Expansión de almacenamiento — Partición NTFS como volumen auxiliar

Con el espacio recuperado, el margen seguía siendo limitado para el desarrollo futuro del proyecto. La máquina de desarrollo cuenta con un disco de mayor capacidad con una partición NTFS administrada desde Windows (/dev/sdb2, 733 GB). Se evaluaron las opciones disponibles:

Opción	Descripción	Riesgo
Redimensionar <code>/dev/sdb3</code>	Ampliar la partición Linux desde el disco no particionado	Alto — requiere sistema en vivo, riesgo de pérdida de datos
Reducir <code>/dev/sdb2</code> desde Linux	Redimensionar la partición NTFS desde Linux para ceder espacio a ext4	Alto — operación no soportada de forma nativa; puede corromper NTFS
Montar <code>/dev/sdb2</code> como volumen adicional	Acceder a los 529 GB disponibles en la partición NTFS sin modificarla	Sin riesgo — operación de solo lectura/escritura sobre NTFS existente

Se eligió la opción sin riesgo: montar permanentemente la partición NTFS con `ntfs-3g`, haciendo disponibles los 529 GB como volumen auxiliar en `/mnt/datos`:

```
# Identificar el UUID de la partición
sudo blkid /dev/sdb2
# /dev/sdb2: TYPE="ntfs" UUID="124A39244A39064F"

# Crear el punto de montaje
sudo mkdir -p /mnt/datos

# Agregar entrada permanente en /etc/fstab
echo "UUID=124A39244A39064F /mnt/datos ntfs-3g \
defaults,uid=1000,gid=1000,umask=022,nofail 0 0" \
| sudo tee -a /etc/fstab

# Montar sin reiniciar
sudo mount -a

# Verificar disponibilidad
df -h /mnt/datos
# Filesystem      Size  Used Avail Use% Mounted on
# /dev/sdb2       733G  204G  529G   28%  /mnt/datos
```

Resultado: 529 GB adicionales disponibles de forma permanente en `/mnt/datos`, sin modificar la tabla de particiones ni arriesgar la integridad del sistema Linux.

Traspaso formal — Onboarding de Diego Vázquez

Fecha de inicio: 28 de septiembre de 2026

En coordinación con la Dra. Montserrat, se inició el traspaso del entorno actualizado al alumno Diego Vázquez, quien continuará con la siguiente fase operativa del proyecto.

Diagnóstico de prerequisites:

Se detectó que Diego contaba con Ubuntu 22.04, versión incompatible con ROS 2 Jazzy Jalisco (que requiere Ubuntu 24.04 como base). Se acordó la creación de una partición dedicada para instalar Ubuntu 24.04,

garantizando compatibilidad nativa con el stack cognitivo del proyecto.

Materiales entregados:

- URL del repositorio oficial: <https://github.com/oscaregop98/xolobot-cognitive-core>
- Enlace a la documentación oficial de instalación de ROS 2 Jazzy
- Instrucción de lectura previa del [README.md](#) maestro antes de ejecutar cualquier comando

Metodología de onboarding:

Se instruyó a Diego a leer primero el README para familiarizarse con la arquitectura del proyecto (opciones Nativa y Docker) y los alias del panel de control antes de proceder con la instalación. Las dudas surgidas durante el proceso de instalación serán monitoreadas y documentadas para robustecer la documentación del repositorio de cara a generaciones futuras de integrantes.

Observaciones y Conclusiones

Observaciones técnicas

1. **La reproducibilidad es una propiedad que debe construirse, no asumirse.** El Acid Test demostró que un entorno que funciona en el equipo del desarrollador no es automáticamente reproducible en otro hardware. Las horas de compilación en [ss-neuro](#), el comportamiento del sistema ante recursos limitados, y la necesidad de seguir una documentación precisa son aspectos que solo se revelan ejecutando la prueba en condiciones reales.
2. **El ecosistema ROS 2 + Gazebo Harmonic tiene un coste de almacenamiento inherentemente alto.** La combinación de librerías de simulación física, modelos 3D, herramientas de visualización y el compilador de C++ produce artefactos que consumen decenas de gigabytes. Esto no es una disfunción del proyecto sino una característica del stack tecnológico, y debe anticiparse al dimensionar cualquier máquina de desarrollo nueva.
3. **La reorganización del repositorio es una inversión en continuidad del proyecto.** Separar el trabajo de migración (historial de errores y correcciones) del código estable en producción reduce la barrera de entrada para nuevos integrantes. El nuevo repositorio [xolobot-cognitive-core](#) es el punto de partida para quien se incorpore al proyecto en fases futuras.
4. **La invalidación de [CMakeCache](#) por actualizaciones de sistema es un patrón recurrente.** El conflicto [libfastcdr 2.2.7 → 2.2.8](#) es un caso específico de un problema general: cuando el gestor de paquetes del sistema operativo actualiza una librería enlazada dinámicamente, los artefactos de compilación de colcon quedan inconsistentes. El procedimiento `rm -rf build/install/ log/ && colcon build` debe tratarse como la respuesta estándar ante cualquier fallo de linking después de un `apt upgrade`.
5. **Montar el volumen NTFS como solución de almacenamiento fue la elección correcta.** Las alternativas de redimensionamiento de particiones conllevaban riesgo de pérdida de datos en una máquina que ya contiene trabajo en curso. El montaje no destructivo permitió ampliar la capacidad de almacenamiento disponible de forma inmediata y reversible.

Conclusiones

Al término de los Meses 4, 5 y 6 del calendario oficial, el proyecto completó su transición de la fase de implementación a la fase de **validación y transferencia**. La arquitectura cognitiva del Xolobot no solo funciona en el equipo de desarrollo original, sino que fue demostrada como reproducible en hardware independiente bajo condiciones de instalación limpia.

Actividad	Resultado	Semanas
Validación Docker en VM (4 problemas)	Todos los problemas confirmados como estables; ciclo completo de arranque verificado	13–15
Reorganización del repositorio	xolobot-cognitive-core en GitHub, documentación maestra, onboarding autónomo posible	16–18
Resolución de conflicto libfastcdr	Rebuild limpio documentado como procedimiento estándar post- apt upgrade	16–18
Acid Test en ss-neuro	Gazebo Harmonic ejecutándose en instalación limpia Ubuntu 24.04; secuencia de agarre completada	19–21
Gestión de almacenamiento	Liberación de 13 GB (97% → 81%); 529 GB auxiliares disponibles en /mnt/datos	22–24
Traspaso a Diego Vázquez	Prerrequisitos diagnosticados, materiales entregados, onboarding iniciado	22–24

El estado del proyecto al cierre del servicio social es el siguiente: la arquitectura cognitiva del Xolobot opera en ROS 2 Jazzy Jalisco con Gazebo Harmonic, su documentación permite la incorporación autónoma de nuevos integrantes, y el proceso de traspaso a la siguiente generación de alumnos ha sido formalmente iniciado con la orientación de la responsable del proyecto.